Session 4: From Data to Deliverable

Business Intelligence & AI

Kerry Back, Rice University

Homework Debrief

Best and Worst Answers

Share Your Best Answer

- What question did you ask?
- What made the answer good?
- Would you trust it without further checking?

Share Your Worst Answer

- What went wrong?
- Was the answer wrong, misleading, or just incomplete?
- What verification would have caught it?

Applying Maker/Checker

Low Stakes

- Internal exploration, brainstorming
- “How many customers does each division have?”
- Quick sanity check: do the numbers look reasonable?

Medium Stakes

- Team presentations, internal reports
- “Revenue per employee by division”
- Verify: check the SQL, confirm date ranges, spot-check totals

High Stakes

- Board decks, regulatory filings
- “Quarterly executive summary for the CEO”
- **Independent verification of every claim**

Full Pipeline Demo

From Questions to Deliverables

In Sessions 1–3, we asked questions and got answers. Today we chain those steps into **complete reporting pipelines** — from raw data to polished, board-ready documents.

> One prompt. Six systems queried. Charts generated. Narrative written. Document assembled. **That is the pipeline.**

The Pipeline Steps

1

Query

Pull from 6 source systems

→

2

Merge

Join and deduplicate

→

3

Compute

Calculate KPIs and variances

→

4

Chart

Generate visualizations

→

5

Narrate

Write analysis text

→

6

Assemble

Format final document

Live Demo: “Prepare a Quarterly Executive Summary”

The prompt

```
Prepare a quarterly executive summary of XYZ Corp performance ---
revenue by division, headcount efficiency, top customer risks, and
supply chain issues. Include charts and a narrative summary.
```

Demo Result: Q4 Executive Summary

Division	Q4 2025	Q3 2025	QoQ Growth	Active Customers
Industrial	$5,891,237	$5,482,103	+7.5%	91
Energy	$3,412,891	$3,187,445	+7.1%	65
Safety	$1,147,892	$1,023,556	+12.1%	37
Total	**$10,452,020**	**$9,693,104**	**+7.8%**	**193**

Safety posted the strongest growth rate at **12.1%**. But Energy has a **34% customer concentration risk** in CAT Energy Services — a single-point-of-failure.

Now Iterate

Draft 1: Data Summary

- Revenue by division, headcount, growth rates
- Comprehensive but flat
- “Summarize Q4 performance”

Draft 2: Traffic-Light Format

- Red / Yellow / Green status by division
- Flags anything >10% off plan
- “Traffic-light format, flag >10% off plan”

Draft 3: Executive Brief

- Leads with the biggest risk
- Ends with 3 recommended actions
- **“One page, lead with risk, end with actions”**

Same data. Same agent. **The prompt was the variable that mattered.**

Three Iterations Compared

Draft	Prompt Addition	Output	Audience
1	“Summarize Q4 performance”	Data dump — no prioritization	Analyst
2	+ “Traffic-light, flag >10% off plan”	Structured dashboard — scannable	Manager
3	+ “One page, lead with risk, end with actions”	Executive brief — board-ready	C-suite

The prompt is not just a question — it is a **specification**. More specific prompts produce more useful deliverables. This is the skill you developed in Session 2, now applied to enterprise data.

How the Agent Loop Works

The Core Loop

1

System Prompt

Schema description: what tables, columns, and relationships exist

→

2

Tool Call

LLM writes SQL and requests execution

→

3

Execute & Return

System runs SQL, sends results back to LLM

→

4

Test & Review

LLM checks results — correct? try again? deliver?

This is a **test-and-review cycle**, not a one-shot generation. The LLM writes code, the system executes it, the LLM reviews the results, and decides: deliver or try again.

The System Prompt Is Everything

What the LLM sees before your question

```
You are a data analyst for XYZ Corp, a $500M B2B distributor.

Available databases:
- sf_opportunities: Salesforce CRM (Industrial division)
  Columns: OpportunityId, AccountName, Amount, CloseDate, Stage...
- legacy_orders: Legacy CRM (Energy division)
  Columns: order_id, customer_name, order_amount, order_date (VARCHAR!)...
- hs_deals: HubSpot CRM (Safety division)
  Columns: deal_id, company_name, amount, close_date...
- wd_employees: Workday HR
  Columns: employee_id, division, department, hire_date, salary...

IMPORTANT: legacy_orders stores dates as MM/DD/YYYY text strings.
```

The Test-and-Review Cycle

What Makes This Different from a Dashboard

- Dashboards run **pre-written** queries
- If the query is wrong, you get wrong answers forever
- Changing the question means changing the code
- No self-correction capability

What the Agent Does

- Writes a **new query** for every question
- Reviews results: empty set? error? retry
- Can rewrite SQL 2-3 times before delivering
- **Self-corrects — but not infallibly**

From Ad Hoc to Automated

The Production Pattern

What You Have Been Doing

- Type a question in a chat interface
- Get an answer in real time
- Useful — but manual and ad hoc
- You still have to remember to ask

What Production Looks Like

- Same agent loop, triggered by a **schedule**
- Monday morning dashboard refresh
- Monthly board deck, weekly sales forecast
- **Error handling + version control + human review gates**

Real-World Benchmark: HPE

“Alfred” — HPE’s AI Agent

HPE’s CFO Marie Myers deployed an AI agent that replaced a **100-slide weekly performance review** that consumed hundreds of person-hours. The agent queries financial systems, generates analysis, and produces executive summaries — the same pipeline you just saw.

40%

faster reporting cycle

25%

lower processing costs

90%

reduction in manual prep

Hands-On: Build Your Own Data Agent

From English to Working Agent

What You Are About to See

- The instructor opens a terminal on the AI+Code Lab
- Types English instructions to Claude Code: “Build a Python script that takes a question, sends it to Claude with a schema description, gets SQL back, runs it, and prints the answer”
- Claude Code generates ~50 lines of Python
- We run it. We ask a question. We get an answer.

> The entire agent — from blank file to working code — takes about **3 minutes**. Then you build one yourself.

Your Turn: Build a Data Agent

Exercise: Build Your Own Agent (12 minutes)

1. **Open the AI+Code Lab** terminal
2. **Tell Claude Code:** “Build a Python script that takes a business question, sends it to Claude with a description of the XYZ Corp data schema, gets back a SQL query, runs it against the data file, and prints the answer.”
3. **Test it** with 2-3 questions:
    - “What is total revenue by division?”
    - “Which customers have the most support tickets?”
    - “What is the average deal size in the Energy division?”
4. **Verify the answers** — check one result against what you know from the XYZ Corp Custom Chatbot

The 50 Lines vs. the 3,000 Lines

50 Lines: What You Just Built

- System prompt with schema
- Tool call mechanism (SQL execution)
- Basic response formatting
- Works for one database, one user
- **Built from English in minutes**

3,000 Lines: Production

- Multiple database connections
- Authentication and access control
- Error handling and retry logic
- Logging, monitoring, audit trail
- Web interface, API, scheduling
- **This is the XYZ Corp Custom Chatbot**

The core is simple. Production is engineering. Understanding the difference helps you evaluate vendors and scope internal builds.

From Agent to Dashboard

Your agent answers questions in a terminal. But the same AI tools can build **interactive dashboards** — web apps with charts, filters, and drill-downs.

> One prompt to Claude Code: “Turn my data agent into an interactive dashboard with charts and filters.” Working app in under **5 minutes**.

Workshop: Your Reporting Candidates

Identify Your Pipelines

Individual Exercise (12 minutes)

For **2-3 recurring reports** at your company:

1. **What is the report?** (weekly status, monthly board deck, quarterly review, etc.)
2. **What systems feed it?** (CRM, ERP, HR, finance)
3. **Who builds it today?** How long does it take?
4. **What would automation look like?** Agent queries sources, generates draft, human reviews

Report	Systems	Owner Today	Time	Automation Potential
e.g., Weekly sales forecast	*Salesforce + NetSuite*	*Sales Ops analyst*	*4 hours/week*	*High — structured data, repeating format*
e.g., Monthly board deck	*All systems*	*FP&A team*	*3 days/month*	*Medium — needs heavy human review*

Where Does the Code Run?

Three Execution Options

Cloud LLM Sandbox

- Code runs on the vendor's servers
- Easiest to set up
- Your data leaves your network
- What we use for the XYZ Corp demo (synthetic data)

Your Own Containers

- Code runs on your servers
- Data stays on-premise
- You manage infrastructure
- Higher control, higher cost

SQL-Only Mode

- LLM only generates SQL, never sees raw data
- Your system executes the SQL locally
- Minimum data exposure
- **Most restrictive but most secure**

Our demo uses synthetic data in the cloud. Production deployments keep data on-premise. Session 5 digs into these tradeoffs in detail.

Homework

Homework for Session 5

1. **Produce one complete deliverable** using the XYZ Corp Custom Chatbot — pick one:
    - **Customer Risk Report:** Combine CRM data with support tickets. Which customers are high-revenue AND high-complaint?
    - **Division P&L Comparison:** Revenue by division from CRMs + headcount from Workday. Revenue per employee.
    - **Board Memo:** Q3 vs Q4 performance across all divisions. Charts, narrative, risk flags. Iterate until it fits on one page.
2. **Pre-reading (~35 min):**
    - Morgan Stanley case study (provided) — a regulated firm's deployment decision
    - AIMultiple: "Cloud LLM vs Local LLMs" — three deployment options
    - "What Does Your AI Actually See?" (provided) — four data security approaches
3. **Optional reading:** a16z "How 100 Enterprise CIOs Are Building and Buying Gen AI" + DUNNIXER vendor evaluation scorecard
4. *Optional:* **Extend your agent.** Go back to the AI+Code Lab and improve the data agent you built today. Ideas: add chart generation, add a web interface, connect a second data source. The more you build

Session 5 Preview

Session 5: “What Does Deployment Look Like?”

- Where does the LLM run? Cloud vs. on-premise vs. hybrid
- What does the AI actually see? Data security approaches
- Build vs. buy vs. extend — the real decision matrix
- Morgan Stanley case study discussion

Come prepared: read the case study and form an opinion.

Deep Dive: Inside an Agent

What Is an Agent?

An agent is a program that:

1. Maintains a conversation with a user
2. Has access to **tools** (functions it can call)
3. **Decides which tools to use** based on the question
4. Chains multiple tool calls before responding

This is different from a script that always runs the same steps. The agent **reasons** about what information it needs and fetches it on the fly.

Why an Agent, Not a Script?

A Script

- Fixed sequence of steps
- Same analysis every time
- Can’t handle follow-up questions
- New analysis = new code

An Agent

- Chooses tools based on context
- Different path for each question
- Conversational — remembers context
- New analysis = just ask

You write the tools once; the agent figures out how to combine them.

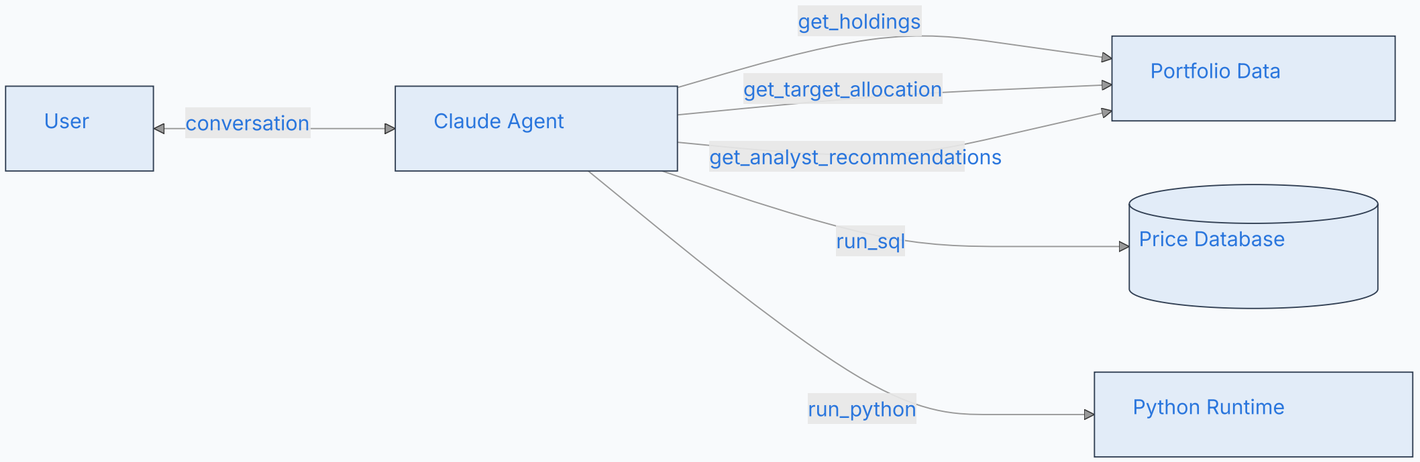
Our Example: Portfolio Review Agent

We'll walk through a **portfolio analyst** that can:

- Look up current holdings and tax lots
- Query a live price database
- Compute returns, correlations, and statistics in Python
- Compare sector weights against targets
- Identify tax-loss harvesting opportunities
- Find the most correlated Strong Buy replacement

The agent chooses its own path. Asking "review the portfolio" triggers different tools than "find tax-loss candidates in energy."

Architecture



Five tools. Three return portfolio data. One executes SQL against a live price database. One runs Python for analytics.

Tools Are JSON Descriptions

Each tool has a `name`, `description`, and `input_schema`. The description tells the agent **when and why** to use it.

```
tools = [
    {
        "name": "get_holdings",
        "description": (
            "Retrieve all portfolio holdings with tax lot detail. "
            "Returns each position's ticker, sector, and individual "
            "lots (date acquired, shares, cost basis per share)."
        ),
        "input_schema": {
            "type": "object",
            "properties": {},
```

The description is the critical part — it tells the agent when to use this tool. A tool with no inputs (like `get_holdings`) acts as a simple lookup.

The SQL and Python Tools

The agent can **write its own queries** and **run its own analytics**:

run_sql

```
Execute a SQL query against the
price database.
Available tables:
  - sep: daily stock prices
  - tickers: stock metadata
Use closeadj for return calculations.
```

Input: a SQL string the agent composes on the fly

run_python

```
Execute Python code and return
printed output.
pandas, numpy, and json are
available.
The namespace persists across calls.
```

Input: Python code — the agent decides **what** to compute

Multi-tool chains: `run_sql` fetches prices → `run_python` computes correlations. The agent decides the sequence.

The Tool Router

```python
def execute_tool(name: str, inputs: dict) -> str:
    """Route a tool call to the appropriate handler."""
    if name == "get_holdings":
        return json.dumps(HOLDINGS)
    elif name == "get_target_allocation":
        return json.dumps(TARGET_ALLOCATION)
    elif name == "get_analyst_recommendations":
        sector = inputs["sector"]
        recs = STRONG_BUY.get(sector, [])
        return json.dumps({"sector": sector, "strong_buy": recs})
```

This is your code — you control what each tool does. The agent only sees the tool's name, description, and the result you return.

The Core Loop

```python
import anthropic

client = anthropic.Anthropic()
messages = [{"role": "user", "content": user_input}]

while True:
    response = client.messages.create(
        model="claude-sonnet-4-6",
        system=SYSTEM_PROMPT,
        tools=tools,
```

The loop keeps running until the agent has all the information it needs. It might call one tool or five.

Handling Tool Calls

```
if response.stop_reason == "tool_use":
    # Save the assistant's message
    messages.append({"role": "assistant", "content": response.content})

    # Execute each tool and collect results
    tool_results = []
    for block in response.content:
        if block.type == "tool_use":
            result = execute_tool(block.name, block.input)
            tool_results.append({
                "type": "tool_result",
```

The `tool_use_id` links each result to the corresponding call. The agent can make **multiple tool calls** in one turn.

The System Prompt

```
SYSTEM_PROMPT = """\
You are a portfolio analyst assistant. You help review a client
portfolio, identify tax-loss harvesting opportunities, and
recommend replacement securities.

When harvesting losses, sell the lots with the largest unrealized
loss first (highest cost basis relative to current price).

When recommending a replacement, compute return correlations
between the stock being sold and each Strong Buy candidate,
```

The system prompt gives the agent its **strategy** — domain expertise that guides tool selection. Compare this to the XYZ Corp system prompt you saw earlier — same pattern, different domain.

Example: Tax-Loss Harvesting

> **You:** Harvest the INTC losses. What should I buy to replace the exposure?

The agent:

1. Identifies INTC lots with the largest unrealized losses
2. Calculates the realized loss
3. Checks if selling moves Technology below target weight
4. Calls `get_analyst_recommendations("Technology")`
5. Calls `run_sql` to fetch daily prices for INTC and each candidate
6. Calls `run_python` to compute return correlations
7. Recommends the most **highly correlated** Strong Buy stock

Seven tool calls across three tools, **none predetermined**. The path depends on what the human asks and what the data shows.

The Agent SDK Simplifies This

Before and After

Manual Loop (~30 lines of plumbing)

```python
client = anthropic.Anthropic()
messages = [{"role": "user", "content": prompt}]

while True:
    response = client.messages.create(
        model="claude-sonnet-4-6",
        system=SYSTEM_PROMPT,
        tools=tools,
        messages=messages,
        max_tokens=4096,
    )
```

Agent SDK (~5 lines)

```python
from claude_agent_sdk import (
    query, ClaudeAgentOptions
)

async for message in query(
    prompt="Review the portfolio",
    options=ClaudeAgentOptions(
        allowed_tools=[
            "Read", "Edit", "Bash",
        ],
    ),
```

- **The loop is gone** — the SDK handles it
- Tool execution is automatic
- Message tracking is automatic
- You focus on **what** the agent does, not **how**
  loops

Why Both Matter

Understanding the Loop

- You saw the raw machinery
- You know what happens at each step
- You can evaluate vendors and debug failures
- The **checker** needs to understand the system

Using the SDK

- Faster to build
- Less code to maintain
- Fewer places for bugs
- Production-ready patterns built in

The Takeaway

- The manual loop teaches you **what agents are**
- The SDK lets you **build them fast**
- Both use the same tools, same system prompt, same model
- The difference is plumbing, not capability

Session 4 Summary

From Data to Deliverable — and You Built the Agent

50

lines of code you built from English

3,000

lines separate prototype from production

90%

manual prep reduction at HPE